Serial Python Minimalis: Tutorial #01-Python Dasar

Ahmad R. T. Nugraha dan Meng E. Ong

ver. 21 April 2026

Seri tutorial "Python Minimalis" ini ditujukan bagi para pelajar fisika secara khusus dan sains secara umum agar dapat "secukupnya" menguasai pemrograman dalam bahasa Python untuk keperluan komputasi numerik. Karena "minimalis", paradigma pemrograman yang ditekankan adalah pemrograman prosedural walaupun Python mendukung teknik lebih lanjut seperti pemrograman berorientasi objek (*object-oriented programming*/OOP).

Sebelum memulai pemrograman numerik, kita perlu menyiapkan kelengkapan "persenjataan" kita. Pemrograman Python untuk keperluan kita ini membutuhkan dua komponen utama: **interpreter** (mesin penerjemah kode) dan **integrated development environment** (IDE) atau tempat menulis kode. Kombinasi yang direkomendasikan adalah *interpreter* **Python** dari situs web resminya (atau dari repositori standar setiap distribusi Linux) dan IDE **Visual Studio Code (VS Code)**. Jika "malas" menginstal di komputer sendiri, kita masih bisa menggunakan lingkungan pemrograman Python berbasis *cloud* seperti `Google Colab`.

1 Instalasi Python dan Kelengkapannya

Untuk pemrograman Python yang profesional, kita tidak disarankan menginstal pustaka (*library*) secara global di sistem komputer karena dapat menyebabkan konflik versi. Praktik terbaik (*best practice*) adalah dengan menggunakan **virtual environment** (`venv`), yakni semacam "ruang isolasi" tempat kita bisa menginstal Python dan paket-paket tertentu (seperti NumPy atau Jupyter) khusus untuk satu proyek saja tanpa mengganggu sistem utama.

Langkah 1: Instalasi Python & VS Code

Pastikan dua komponen dasar berikut ini terinstal:

1. **Python** dari `python.org` untuk pengguna Windows.
   Penting: Saat instalasi, pastikan mencentang opsi **"Add Python to PATH"**. Sementara itu, bagi pengguna Linux, tergantung distribusi yang digunakan,

biasanya sudah ada instalasi Python dasar. Jika belum, misalnya untuk distribusi berbasis Debian/Ubuntu, kita bisa jalankan:

```
sudo apt install python3 python3-venv
```

2. **Visual Studio Code (VS Code).**
   Unduh (*download*) *installer* VS Code dari `code.visualstudio.com` dan lakukan proses instalasi standar sesuai sistem operasi yang digunakan.

Langkah 2: Membuat Folder Proyek & Virtual Environment

Kita akan membuat folder khusus untuk belajar Python dan membuat lingkungan virtual di dalamnya agar rapi.

1. Buat folder baru di komputer, misal bernama `PythonMinimalis`.
2. Buka folder tersebut menggunakan VS Code (*File* > *Open Folder*).
3. Buka Terminal di dalam VS Code dengan menekan `Ctrl + ‘` (tanda *backtick* di sebelah angka 1) atau menu *Terminal* > *New Terminal*.
4. Ketik perintah berikut di terminal untuk membuat `venv`:

   ```
   python -m venv .venv
   ```

   Jika sukses, folder baru bernama `.venv` akan muncul di panel kiri VS Code yang bertanda “Explorer”.

Langkah 3: Aktivasi Environment

Sebelum menginstal paket, kita harus "masuk" ke dalam lingkungan virtual tersebut. Untuk pengguna Linux/macOS, pada terminal bisa eksekusi:

```
source .venv/bin/activate
```

Sementara itu, untuk pengguna Windows, di terminal VS Code (biasanya PowerShell), ketik dan enter:

```
.venv\Scripts\Activate.ps1
```

Masalah Umum di Windows: UnauthorizedAccess

Jika kita mendapat pesan merah: *"...cannot be loaded because running scripts is disabled on this system"*, fitur keamanan Windows telah memblokir eksekusi skrip. Solusinya, jalankan perintah berikut di terminal untuk mengizinkan skrip lokal:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Setelah itu, jalankan perintah aktivasi (`.venv\Scripts\Activate.ps1`) sekali lagi.

Jika berhasil, di sebelah kiri baris perintah akan muncul tanda kurung hijau (atau warna lain) dengan label semacam ini: `(.venv)`.

Langkah 4: Instalasi Paket Ilmiah

Setelah tanda `(.venv)` muncul (artinya kita sudah berada di dalam lingkungan virtual atau ruang isolasi), saatnya menginstal alat tempur kita, di antaranya:

- **numpy**: komputasi numerik array/matriks;
- **scipy**: paket komputasi tingkat lanjut yang siap pakai (integral, optimasi, dll.);
- **matplotlib**: untuk membuat grafik/plot; dan
- **jupyter**: untuk membuat *notebook* interaktif.

Jalankan perintah berikut di terminal:

```
pip install numpy scipy matplotlib jupyter
```

Tunggu hingga proses pengunduhan dan instalasi selesai.

Langkah 5: Menghubungkan VS Code dengan venv

Agar VS Code menggunakan Python yang ada di dalam `.venv` (bukan Python global), kita perlu melakukan konfigurasi yang tepat.

A. Format Jupyter Notebook (`*.ipynb`)

1. Buka atau buat file baru dengan akhiran `.ipynb` (misal: `catatan.ipynb`).
2. Klik tombol kernel di pojok kanan atas editor (biasanya tertulis *Select Kernel* atau *Python 3...*).
3. Pilih **Select Another Kernel...** → **Python Environments**.
4. Cari pilihan yang memiliki label `Star` (⋆) atau yang mengarah ke folder `.venv` kita. *Contoh:* `Python 3.10.0 ('.venv': venv)`.

B. Format Skrip Python Biasa (`*.py`)

1. Buka atau buka file berakhiran `.py`.
2. Klik indikator versi Python di pojok kanan bawah jendela status bar VS Code.
3. Pilih *interpreter* yang mengarah ke `./.venv/Scripts/python.exe` (untuk Windows) atau `.venv` (untuk Linux/macOS).

Sekarang lingkungan kerja kita sudah ala profesional, terisolasi, dan siap untuk komputasi berat tanpa mengganggu sistem operasi utama.

2 Python sebagai Kalkulator

Di antara kegunaan paling sederhana pemrograman Python adalah sebagai kalkulator canggih. Kita dapat menjumlahkan, mengalikan, mengurangi angka atau bilangan, dan semacamnya. Ada tiga jenis bilangan utama dalam Python:

- **Integer** (bilangan bulat) diwakili oleh tipe `int`.
- **Bilangan riil** diwakili oleh tipe `float`. Nama `float` merujuk pada *floating-point numbers* (bilangan titik kambang), yaitu representasi perkiraan dari bilangan riil yang digunakan oleh Python (dan sebagian besar bahasa komputer modern lainnya).
- **Bilangan kompleks** (yang memiliki bagian riil dan imajiner) diwakili oleh tipe `complex`.

Dalam sesi Python interaktif seperti Jupyter Notebook, angka-angka dari tipe bilangan tersebut dapat digabungkan menggunakan operator dalam satu ekspresi yang dievaluasi dan hasilnya dikembalikan ke *prompt* (baris) keluaran. Contohnya:

```
1 + 2
```

Output:

```
3
```

```
10 / 4
```

Output:

```
2.5
```

```
2356 * 911
```

Output:

```
2146316
```

(**Catatan:** Pada mode interaktif, untuk dapat memunculkan "output" yang relevan, bisa dengan klik tombol semacam ▶`Run` yang ada pada editor/lingkungan pemrograman masing-masing atau dengan jalan pintas variasi tombol kibor tertentu, seperti `CTRL+Enter` atau `SHIFT+Enter`.)

Untuk membuat kode lebih mudah dipahami, ada baiknya kita rutin menambahkan komentar. Bentuk komentar singkat dapat berupa segala sesuatu pada satu baris setelah karakter # yang akan diabaikan oleh *interpreter* Python. Komentar ini berguna sebagai catatan yang dapat dibaca kita sendiri atau orang lain sehingga konteks kode akan lebih jelas. Contohnya:

```
# Entalpi fusi molar es: konversi dari kJ.mol-1 ke J.mol-1.
6.01 * 1000
```

Output:

```
6010.0
```

```
6.518 / 1013.25 * 760 # tekanan atmosfer di Mars, dalam Torr
```

Output:

```
4.888902047865778
```

Operator dan Urutan Operasi

Operator aljabar dasar tercantum dalam Tabel 1. Dalam penggunaannya, penting bagi kita untuk memperhatikan urutan prioritas operasi (Tabel 2), yakni urutan operator tersebut diterjemahkan dalam sebuah ekspresi. Contohnya:

```
1 + 3 * 4
```

Output:

```
13
```

Di sini, $3 * 4$ dievaluasi terlebih dahulu karena operator perkalian, $*$, memiliki urutan prioritas yang lebih tinggi daripada + dan $*$. Hasilnya, 12, kemudian ditambahkan pada 1. Jika operator memiliki urutan prioritas yang sama, bagian-bagian dari ekspresi umumnya dievaluasi dari kiri ke kanan (kecuali pemangkatan atau eksponensial yang dievaluasi dari kanan ke kiri). Urutan prioritas ini dapat dianulir dengan menggunakan tanda kurung biasa:

```
6 / 3 ** 2      # sama dengan 6 / 9
```

Output:

```
0.6666666666666666
```

```
(6 / 3) ** 2    # sama dengan 2 ** 2
```

Output:

```
4.0
```

Tabel 1: Operator aritmetika dasar Python.

Simbol	Operasi
+	Penjumlahan
-	Pengurangan
*	Perkalian
/	Pembagian riil (*float*)
//	Pembagian bulat khusus
%	Modulus (sisa bagi)
**	Pemangkatan

Tabel 2: Urutan prioritas operator aritmetika Python.

Operator	Tingkat Prioritas
**	(prioritas tertinggi)
*, /, //, %	(pertengahan & setara)
+, -	(prioritas terendah)

Pembagian

Perhatikan bahwa ekspresi di atas menghasilkan bilangan *floating point* meskipun kita mengoperasikan bilangan bulat. Kejadian tersebut dikarenakan operator pembagian, `/`, selalu mengembalikan `float`, bahkan ketika hasilnya adalah bilangan bulat.

Perlu diketahui bahwa ada operator pembagian khusus integer, `//`, yang mengembalikan hasil bagi bulat dari pembagian ("berapa kali angka kedua cukup dekat ke angka pertama"). Operator yang terkait, yakni `modulus`, `%`, memberikan sisa pembagiannya. Mari kita lihat contoh-contoh berikut ini.

```
7 / 3
```

Output:

```
2.3333333333333335
```

```
7 // 3
```

Output:

```
2
```

```
7 % 3
```

Output:

```
1
```

Sebagai catatan tambahan, amati hasil dari `7 / 3`. Nilai tepatnya, $2\frac{1}{3}$, tidak dapat direpresentasikan secara eksak ketika Python memformat bilangan riil (*floating-point numbers*) yang representasinya dalam komputer memiliki presisi terbatas (sekitar 1 dalam 10^{16}). Nilai `float` terdekat dengan jawaban yang dapat direpresentasikanlah yang dikembalikan.

Pustaka Matematika (`math` dan `numpy`)

Selain operator aljabar dasar, terdapat banyak fungsi matematika serta konstanta seperti π dan e yang disediakan oleh pustaka `math` dalam Python. Pustaka ini adalah modul bawaan yang disediakan di setiap instalasi Python (tidak perlu menginstal paket tambahan), tetapi harus diimpor dengan perintah: `import math`.

Fungsi-fungsi tersebut (sebagian tercantum dalam Tabel 3) kemudian dapat digunakan dengan menambahkan awalan `math.` Contohnya:

```
import math
math.sin(math.pi/4)
```

Output:

```
0.7071067811865475
```

Cara seperti di atas adalah contoh pemanggilan fungsi. Fungsi `math.sin` diberikan sebuah argumen (di sini, angka $\pi/4$) di dalam tanda kurung. Ekspresi tersebut kemudian mengembalikan hasil perhitungannya.

Paket **NumPy**, yang awalnya bukan bawaan Python (perlu diinstal secara terpisah), menyediakan semua fungsionalitas `math` dan berbagai tambahan fitur numerik. Oleh karena itu, kita akan lebih sering menggunakannya daripada `math`. Biasanya NumPy diimpor dengan alias `np`, seperti pada contoh berikut:

```
import numpy as np
1 / np.sqrt(2)
```

Output:

```
0.7071067811865475
```

Tabel 3: Beberapa fungsi dan konstanta yang disediakan oleh modul `math`. Sudut diasumsikan dalam radian.

Fungsi/Konstanta	**Deskripsi/Nilai Matematika**
`math.pi`	π
`math.e`	e
`math.sqrt(x)`	$\sqrt{x}$
`math.exp(x)`	e^x
`math.log(x)`	$\ln x$
`math.log10(x)`	$\log_{10} x$
`math.sin(x)`	$\sin(x)$
`math.cos(x)`	$\cos(x)$
`math.tan(x)`	$\tan(x)$
`math.asin(x)`	$\arcsin(x)$
`math.acos(x)`	$\arccos(x)$
`math.atan(x)`	$\arctan(x)$
`math.hypot(x, y)`	jarak Euklides (*Euclidean norm*), $\sqrt{x^2+y^2}$
`math.comb(n, r)`	Koefisien binomial, $\binom{n}{r} \equiv {}^nC_r$
`math.degrees(x)`	Konversi x dari radian ke derajat
`math.radians(x)`	Konversi x dari derajat ke radian

Meskipun fungsi trigonometri dalam `math` dan NumPy menggunakan radian alih-alih derajat, ada beberapa metode praktis untuk saling konversi antara keduanya:

```
np.degrees(np.pi/2)
```

Output:

```
90.0
```

```
np.sin(np.radians(30))
```

Output:

```
0.49999999999999994
```

Perhatikan kembali presisi yang terbatas pada contoh tersebut. Kita tahu nilai eksak dari $\sin(30^\circ)$ adalah 0.5, tetapi representasi oleh komputer dapat berupa angka 0.49999999999999994 seperti pada contoh.

Fungsi `math.log` dan `np.log` memberikan logaritma natural (basis e). Ada pula varian `math.log10` dan `np.log10` yang terpisah:

```
np.log(10)
```

Output:

```
2.302585092994046
```

```
1 / np.log10(np.e)
```

Output:

```
2.302585092994046
```

Fungsi Bawaan Lainnya

Ada beberapa fungsi bawaan yang berguna (yaitu, yang tidak memerlukan paket seperti `math` atau NumPy untuk diimpor):

- `abs` mengembalikan nilai absolut dari argumennya.
- `round` membulatkan angka ke presisi tertentu dalam digit desimal (atau ke bilangan bulat terdekat jika tidak ada presisi yang ditentukan).

3 Pendefinisian Bilangan

Berbeda dengan beberapa bahasa pemrograman lain, Python tidak mengharuskan pengguna/pemrogram untuk mendeklarasikan tipe bilangan sebelum digunakan. Angka-angka yang terlihat oleh *interpreter* sebagai bilangan bulat akan diperlakukan sebagai objek `int`, sementara yang tampak seperti bilangan riil akan menjadi objek `float`. Namun, tipe data ini adalah bilangan tanpa dimensi. Jika ada besaran fisis yang diwakili tipe-tipe data tersebut, pemrogram bertanggung jawab untuk menjaga sendiri satuannya.

Bilangan bulat dalam Python bisa sebesar apapun yang diizinkan memori komputer. Untuk mendefinisikan bilangan bulat yang sangat besar, akan sangat mudah memisahkan kelompok digit dengan karakter garis bawah, '_'.

```
# Konstanta Avogadro (mol-1): nilai eksak berdasarkan definisi.
602_214_076_000_000_000_000_000
```

Output:

```
602214076000000000000000
```

Bilangan titik kambang (*floating point*) dapat ditulis dengan titik desimal '.', dan boleh disertai pengelompokan digit opsional untuk memperjelas:

```
# Konstanta Gas (J.K-1.mol-1): nilai eksak berdasarkan definisi.
8.31_446_261_815_324
```

Output:

```
8.31446261815324
```

Selain itu, dalam notasi ilmiah, kita bisa menggunakan karakter `e` (atau `E`) yang memisahkan mantisa (digit signifikan) dan eksponen:

```
# Konstanta Boltzmann (J.K-1): nilai eksak berdasarkan definisi.
1.380649e-23
```

Output:

```
1.380649e-23
```

Bilangan kompleks dapat ditulis sebagai jumlah dari bagian riil dan imajiner. Bagian imajiner dalam Python ditandai dengan akhiran `j` (bukan `i`) mengikuti konvensi dalam *engineering*:

```
1 + 4j
```

Output:

```
(1+4j)
```

Cara lain, kita bisa secara eksplisit memberikan sepasang nilai pada fungsi `complex`:

```
complex(-2, 3)
```

Output:

```
(-2+3j)
```

Dalam bilangan kompleks, bagian riil dan imajiner direpresentasikan dalam bentuk titik kambang *floating point* dan dapat diperoleh secara terpisah menggunakan atribut `.real` dan `.imag`. Fungsi bawaan `abs` mengembalikan besaran (magnitudo) bilangan kompleks:

```
(3 + 4j).real
```

Output:

```
3.0
```

```
(3 + 4j).imag
```

Output:

```
4.0
```

```
abs(3 + 4j)
```

Output:

```
5.0
```

4 Variabel

Ketika memecahkan suatu masalah numerik tertentu, kita perlu menyimpan angka-angka dalam program sehingga dapat digunakan berulang kali dan dirujuk dengan nama yang mudah. Untuk mencapai tujuan tersebut, kita perlu mendefinisikan "variabel". Dalam Python, variabel dapat dianggap sebagai label yang disematkan pada objek (misalnya suatu bilangan `int` atau `float`). Ada beberapa aturan penting terkait penamaan variabel:

- Nama variabel boleh mengandung huruf, angka, dan karakter garis bawah (sering digunakan untuk mengindikasikan subskrip).
- Nama variabel tidak boleh diawali dengan angka.
- Variabel tidak boleh memiliki nama yang sama dengan salah satu dari kata-kata kunci tercadang (*reserved keywords*) yang dikhususkan dalam bahasa Python, seperti dapat dilihat pada Tabel 4.

Sebagian besar editor kode modern sudah memiliki fitur sorotan sintaksis yang akan memberikan peringatan saat ada kata-kata kunci khusus yang digunakan. Perhatikan perbedaan antara pemberian nilai atau penugasan (*assignment*) yang valid:

```
Konstanta Avogadro (mol-1): nilai eksak berdasarkan definisi.
N_A = 602_214_076_000_000_000_000_000
```

dan yang invalid:

```
import = 0
```

Output:

```
  File "/tmp/ipython-input-2269274265.py", line 1
    import = 0
           ^
SyntaxError: invalid syntax
```

Upaya pemberian nilai ke variabel bernama `import` gagal dilakukan (muncul `SyntaxError`) karena `import` adalah salah satu kata kunci tercadang. Kata kunci ini merupakan bagian dari sintaksis Python yang digunakan untuk mengimpor modul, seperti yang sudah pernah kita lakukan.

Pada praktiknya, kata kunci tercadang jarang menjadi nama variabel yang mungkin dipilih, dengan pengecualian `lambda` (belakangan akan kita lihat), yang bisa dipilih

untuk merepresentasikan panjang gelombang (*wavelength*). Namun, kita sarankan menggunakan nama lain seperti `lam` dalam kasus ini. Tabel 4 juga memuat tiga kata kunci khusus yang tidak dapat diubah: `True` dan `False`, yang mewakili konsep logika Boolean, serta `None`, yang digunakan untuk menyatakan nilai kosong atau ketiadaan.

Tabel 4: Kata kunci tercadang (*reserved keywords*) dalam Python 3.

and	as	assert	async	await
break	class	continue	def	del
elif	else	except	finally	for
from	global	if	import	in
is	lambda	nonlocal	not	or
pass	raise	return	try	while
with	yield	False	True	None

Nama variabel yang dipilih dengan baik dapat membuat kode Python sangat jelas dan ekspresif:

```
# Konstanta Boltzmann (J.K-1): nilai eksak berdasarkan definisi.
k_B = 1.380649e-23
R = N_A * k_B   # konstanta gas (J.K-1.mol-1)
```

Pada pernyataan terakhir, sisi kanan tanda =, yaitu ekspresi $N_A * k_B$, dievaluasi terlebih dahulu dan nama variabel `R` disematkan pada hasil perhitungan ini.

Kita bisa juga memodifikasi nilai yang terkait dengan nama variabel:

```
n = 1000
n = n + 1
```

Ekspresi $n = n + 1$ tidaklah merepresentasikan persamaan matematika analitik yang valid karena mustahil untuk diselesaikan. Namun, dalam "cara pikir" komputer, pernyataan ini merupakan instruksi untuk mengambil nilai n yang sebelumnya telah ditetapkan, kemudian tambahkan angka satu padanya, dan akhirnya ditetapkan ulang dengan memberi nama n kembali pada hasilnya. Nilai sebelumnya yang sebesar 1.000 langsung "dilupakan". Memori komputer yang digunakan untuk penyimpanannya dibebaskan dan dapat digunakan untuk keperluan lain. Ekspresi seperti pada contoh sangat umum dalam pemrograman apapun sehingga beberapa bahasa pemrograman, termasuk Python, menyediakan jalan pintas (disebut "penugasan yang diperluas" atau *augmented assignment*):

```
n = 1000
n += 1
```

Cara penulisan di atas akan memberikan hasil yang sama dengan sebelumnya. Ada sintaksis serupa untuk operator lain, seperti pengurangan (-=) dan perkalian (*=).

Jalan pintas lain yang berguna dan difasilitasi Python adalah penggunaan nilai yang dipisahkan koma untuk menetapkan beberapa variabel sekaligus, misalnya:

```
a, b, c = 42, -1, 0.5
```

Untuk selanjutnya, kita akan sebaik-baiknya memberikan nama variabel yang bermakna pada objek agar kode lebih bersifat "*self-documenting*" dan kita bisa meminimalkan penggunaan komentar penjelas.

5 Batasan dan Perangkap

Pembagian dengan nol selalu berbahaya dalam operasi matematika apapun. Peringatan *Exception* (kesalahan Python) dapat "dimunculkan" dalam kasus ini, dan eksekusi kode dihentikan. Bentuk pesan kesalahan dapat bervariasi, tergantung perhitungan mana yang memunculkan *Exception* tersebut. Sebagai contoh:

```
1/0
```

Output:

```
ZeroDivisionError         Traceback (most recent call last)
/tmp/ipython-input-2354412189.py in <cell line: 0>()
----> 1 1/0
ZeroDivisionError: division by zero
```

NumPy sedikit lebih toleran terhadap pembagian dengan nol, dan hanya mengeluarkan peringatan:

```
x = 0
y = np.sin(x) / x
```

Output:

```
/tmp/ipython-input-1162324160.py:4: RuntimeWarning: invalid value
 ↪  encountered in scalar divide
  y = np.sin(x) / x
```

Coba eksekusi lagi variabel y:

```
y
```

Output:

```
nan
```

Pada kasus ini, `y` telah ditetapkan ke nilai *floating-point* khusus berlabel `nan`, yang merupakan singkatan dari "*Not a Number*" (bukan angka). Label ini menandakan bahwa hasil perhitungan tidak terdefinisi. Apa yang selanjutnya bisa kita lakukan dengan informasi ini akan terserah kita. Namun, perlu diketahui bahwa tidak ada jalan kembali dari `NaN`. Operasi apa pun yang dilakukan pada nilai ini hanya akan mengembalikan `NaN` lain dan tidak ada yang sama dengan `NaN` (bahkan tidak sama dengan dirinya sendiri).

Beberapa perilaku, sebagian besar yang dihasilkan dari aritmetika *floating-point*, bisa mengejutkan bagi pengguna baru:

```
np.tan(np.pi/2)
```

Output:

```
1.633123935319537e+16
```

Nilai eksak secara matematis untuk $\tan(\pi/2)$ adalah $+\infty$, tetapi karena `np.pi` tidak merepresentasikan π secara eksak, nilai yang dikembalikan hanyalah angka yang sangat besar (orde `e+16` pada kasus ini), bukan tak terhingga.

Bilangan *floating-point*, karena disimpan dalam jumlah memori tetap (8 byte untuk *double-precision floating point*), memiliki magnitudo maksimum (dan minimum). Upaya evaluasi ekspresi yang menghasilkan angka terlalu besar untuk direpresentasikan (sekitar 1.8×10^{308}) akan memberikan "overflow":

```
np.exp(1000)
```

Output:

```
/tmp/ipython-input-2006519253.py:1: RuntimeWarning: overflow encountered in
↪  exp
  np.exp(1000)
np.float64(inf)
```

Pada contoh di atas, kondisi *overflow* telah mengembalikan nilai khusus `inf`, yang berarti tak terhingga (meskipun e^{1000} tidak tak terhingga, hanya sangat besar).

Hal yang sebalknya, yakni *underflow*, dapat terjadi ketika sebuah angka memiliki nilai absolut yang terlalu kecil untuk direpresentasikan dalam bentuk *double-precision floating point* (kurang dari sekitar 2.2×10^{-308}). Pada kasus ini, kita umumnya tidak akan diperingatkan:

```
np.exp(-1000)
```

Output:

```
0.0
```

Studi Kasus

Pada bagian ini, kita berikan beberapa contoh kasus dan solusinya untuk menunjukkan penggunaan Python sebagai kalkulator dari masalah tertentu.

Kasus 1 Stoikiometri Air

Berapa banyak molekul air yang ada dalam segelas air 250 mL? Gunakan rapat massa, $\rho(H_2O(l)) = 1$ g cm^{-3} dan massa molar, $M(H_2O) = 18$ g mol^{-1}.

Solusi

1 mL adalah volume yang sama dengan 1 cm^3 sehingga kita bisa tetap menggunakan satuan tersebut. Kita definisikan beberapa variabel terlebih dahulu:

```
# Konstanta Avogadro, dalam mol-1 dinyatakan hingga 4 angka penting.
N_A = 6.022e23
# Volume air yang sedang ditinjau, dalam cm3.
V = 250
# Rapat massa air, dalam g.cm-3.
rho = 1
# Massa molar H2O, dalam g.mol-1.
M_H2O = 18
```

Air di gelas kita bermassa $m = \rho V$ dan mengandung $n = m/M(H_2O)$ mol air.

```
# Massa air, dalam g.
m = rho * V
# Jumlah zat air, dalam mol.
n = m / M_H2O
n
```

Output:

```
13.88888888888889
```

Jumlah molekul air kemudian adalah $N = nN_A$.

```
n * N_A
```

Output:

```
8.363888888888889e+24
```

Hasil terakhir ini memberikan jawaban $N = 8.364 \times 10^{24}$ molekul.

Kasus 2 Laju Suara dalam Medium

Laju suara c dalam gas dengan massa molar M pada suhu T adalah

$$c = \sqrt{\frac{\gamma RT}{M}},$$

dengan R adalah konstanta gas. Untuk udara, diketahui indeks adiabatik $\gamma = 7/5$. Perkirakan laju suara di udara pada (a) 25°C dan (b) −20°C. Gunakan $M = 29\ \mathrm{g\ mol^{-1}}$.

Solusi

Kita memiliki dua suhu untuk dihitung. Untuk menghindari pengulangan, definisikan faktor f terlebih dahulu:

$$c = f\sqrt{T} \quad \text{dengan} \quad f = \sqrt{\frac{\gamma R}{M}}.$$

Jika kita tetap menggunakan satuan SI (perhatikan bahwa ini berarti menyatakan M dalam $\mathrm{kg\ mol^{-1}}$), kita harus membuat c keluar dalam satuan $\mathrm{m\ s^{-1}}$. Secara eksplisit:

$$[c] = \sqrt{\frac{[\cdot][\mathrm{J\ K^{-1} mol^{-1}}][\mathrm{K}]}{[\mathrm{kg\ mol^{-1}}]}} = \sqrt{\frac{[\mathrm{J}]}{[\mathrm{kg}]}} = \sqrt{\frac{[\mathrm{kg\ m^2 s^{-2}}]}{[\mathrm{kg}]}} = \mathrm{m\ s^{-1}}.$$

```python
import numpy as np
# Konstanta gas dalam J.K-1.mol-1 (4 a.p.).
R = 8.314
# Massa molar rata-rata udara, dalam kg.mol-1.
M = 29 / 1000
# Rasio kapasitas panas C_p / C_V (indeks adiabatik) gas diatomik.
gamma = 7 / 5

# Faktor f dalam m.s-1.K-1/2.
f = np.sqrt(gamma * R / M)

# Konversi suhu pertama dari degC ke K.
T = 25 + 273
f * np.sqrt(T)
```

Output:

```
345.84234000181505
```

```
# Konversi suhu kedua dari degC ke K.
T = -20 + 273
f * np.sqrt(-20 + 273)
```

Output:

```
318.66200881509076
```

Jadi, pada 25°C laju suara adalah 346 m s^{-1}, sedangkan pada −20°C, lajunya 319 m s^{-1}.

Kasus 3 Gerak Jatuh dengan Kecepatan Awal

Sebuah batu dilemparkan vertikal ke bawah dari puncak tebing setinggi $h = 200$ m dengan besar kecepatan awal $v_0 = 5$ m/s. Berapa lama waktu (t) yang dibutuhkan batu tersebut untuk mencapai tanah? Gunakan percepatan gravitasi $g = 9.81$ m/s^2.

Solusi

Posisi batu y pada waktu t dapat ditelusuri dengan persamaan kinematika:

$$y(t) = h - v_0 t - \frac{1}{2} g t^2$$

Batu mencapai tanah ketika $y(t) = 0$, sehingga kita mendapatkan persamaan:

$$h = v_0 t + \frac{1}{2} g t^2 \quad \Rightarrow \quad \frac{1}{2} g t^2 + v_0 t - h = 0,$$

yang merupakan persamaan kuadrat dalam bentuk $at^2 + bt + c = 0$, dengan $a = \frac{1}{2}g$, $b = v_0$, dan $c = -h$.
Kita selesaikan t dengan rumus abc (rumus kuadratik):

$$t = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Karena waktu t harus positif ($t > 0$), kita ambil akar positifnya:

$$t = \frac{-v_0 + \sqrt{v_0^2 - 4(\frac{1}{2}g)(-h)}}{2(\frac{1}{2}g)} = \frac{-v_0 + \sqrt{v_0^2 + 2gh}}{g}$$

```python
# Koefisien persamaan kuadrat: (1/2*g)t^2 + (v0)t - h = 0
a = 0.5 * g
b = v0
c = -h

# Menghitung t menggunakan rumus abc (akar positif)
t_exact = (-b + np.sqrt(b**2 - 4 * a * c)) / (2 * a)
t_exact
```

Output:

```
5.895662701892837
```

Jadi, waktu yang dibutuhkan batu untuk mencapai tanah adalah sekitar 5.90 detik.

Kasus 4 Statistik Maxwell-Boltzmann

Dalam atmosfer matahari ($T \approx 5800\,\mathrm{K}$), atom hidrogen dapat berada pada keadaan dasar ($n = 1$) maupun keadaan tereksitasi. Hitunglah rasio populasi atom hidrogen pada tingkat energi eksitasi pertama ($n = 2$) dibandingkan dengan keadaan dasar ($n = 1$). Diketahui:

- Perbedaan energi antara $n = 1$ dan $n = 2$ adalah $\Delta E = 10.2$ eV.
- Konstanta Boltzmann $k_B = 8.617 \times 10^{-5}$ eV/K.
- Faktor degenerasi untuk tingkat n adalah $g_n = 2n^2$.

Rumus distribusi Boltzmann:

$$\frac{N_2}{N_1} = \frac{g_2}{g_1} e^{-\Delta E/(k_B T)}$$

Solusi

Pertama, kita tetapkan variabel untuk konstanta fisika dan parameter yang diketahui. Perhatikan bahwa energi diberikan dalam satuan elektron-volt (eV) sehingga sebaiknya kita menggunakan konstanta Boltzmann dalam satuan yang sesuai (eV/K) agar satuan saling menghilangkan di eksponen.

```python
import numpy as np
# Konstanta Boltzmann dalam eV/K
kB = 8.617e-5
# Parameter Fisika
T = 5800          # Suhu permukaan matahari (K)
dE = 10.2         # Perbedaan energi (eV)
# Faktor degenerasi g_n = 2n^2
g1 = 2 * (1**2)   # g1 = 2
g2 = 2 * (2**2)   # g2 = 8
```

Rasio populasi ditentukan oleh faktor statistik (rasio degenerasi) dikalikan dengan faktor Boltzmann (eksponensial).

$$\text{Rasio} = \frac{g_2}{g_1} \times \exp\left(-\frac{\Delta E}{k_B T}\right)$$

```python
# Menghitung rasio N2/N1
# Perhatikan tanda kurung pada penyebut (kB * T)
rasio = (g2 / g1) * np.exp(-dE / (kB * T))
rasio
```

Output:

```
5.50763970425776e-09
```

Hasilnya sekitar 5.5×10^{-9}, yang berarti hanya satu dari setiap ~200 juta atom hidrogen yang berada dalam keadaan tereksitasi $n = 2$ di permukaan matahari.

Hati-hati perangkap matematis! Ada kesalahan umum yang sering terjadi saat menerjemahkan rumus fisika seperti $e^{-E/kT}$ ke dalam kode Python satu baris, yaitu masalah urutan prioritas operator.

```python
# SALAH!
# Python akan mengevaluasi (-dE / kB) terlebih dahulu, lalu hasilnya
↪  DIKALIKAN dengan T
rasio_salah = (g2 / g1) * np.exp(-dE / kB * T)
rasio_salah
```

Output:

```
0.0
```

Kode di atas salah karena secara efektif menghitung $e^{(-\Delta E/k_B)\times T}$, menghasilkan pangkat negatif yang sangat besar (sehingga hasilnya *underflow* menjadi 0). Pastikan kita selalu menggunakan tanda kurung untuk penyebut yang terdiri dari perkalian: `np.exp(-dE / (kB * T))`.